AI-Powered Telecom Network Operations Assistant — High-Level Architecture

From the 5G NetOps dataset (100K records) to a validated, grounded troubleshooting answer

① OFFLINE DATA & INDEXING PIPELINE (python -m data.ingest / data.build_index / data.train_models)

Data Source
- Hugging Face
- fenar/5g_netops
- 100K rows x 27 cols (CSV.xz)
- KPIs, alarms, config,
- security, weather,
- city / state / season

Ingestion & Cleaning
- dedupe, type coercion
- median imputation
- physical-bound clipping
- header encoding fix
- categorical normalisation
- case_id = NOC-######

Feature Engineering
- fleet percentile thresholds (P75/P90)
- 21 KPI conditions (degraded/severe)
- 8-class incident taxonomy
- risk score 0-100 -> P1..P4
- row -> incident narrative text
- (record = 1 chunk)

Indexing (Knowledge Layer)
- Dense: all-MiniLM-L6-v2 (384-d)
- -> FAISS IndexFlatIP (cosine)
- Sparse: BM25 (uni+bi-grams)
- on keyword field (scipy CSC)
- SOP playbook (9 docs) embedded
- metadata columns for filters

ML Training
- Fault-risk: HistGradientBoosting
- regressor + high-risk classifier
- + early-warning variant
- Performance: throughput, latency,
- call-drop regressors
- metrics -> model_metrics.json

Artifact store (data/artifacts): cases_indexed.parquet · incidents.faiss · bm25.joblib · playbook_emb.npy · thresholds.json · models.joblib · model_metrics.json · feedback.db

② ONLINE QUERY PATH (FastAPI microservice + multi-agent orchestrator)

NOC Engineer
- natural-language
- question and/or
- live KPI readings

Streamlit UI
- Ask / Prioritise
- Predictor (what-if)
- Monitoring
- Evaluation
- Feedback form

REST/JSON

FastAPI Layer
- /search /classify
- /recommend /prioritize
- /analyze
- /predict/risk
- /predict/performance
- /feedback /metrics
- Pydantic schemas
- X-API-Key (optional)
- request-id + timing

Input Guardrails
- length / charset
- prompt-injection
- patterns -> 422
- PII redaction
- (email, phone,
- IMSI, card)
- domain relevance
- filter validation

indexes *models*

Multi-Agent Orchestrator (supervisor + shared blackboard context, full trace)

1 Query Understanding
- lexicon symptoms
- severity, state/city/
- season/weather
- (+LLM refine)

2 Retrieval (RAG)
- BM25 + dense FAISS
- RRF fusion (alpha)
- metadata filters
- symptom re-rank

3 Classification
- rules + k-NN vote
- of similar cases
- confidence
- (+LLM tie-break)

4 Risk Prediction
- case-based KPI profile
- fault-risk + perf. ML
- risk score -> P1..P4

5 Root Cause & Impact
- KPI z-score deviations
- ML driver ablation
- case prevalence
- users / SLA / MTTR

6 Recommendation
- SOP RAG + cases
- grounded LLM / SOP
- feedback-ranked
- output guardrails

7 LLM-as-Judge
- groundedness, correct-
- ness, actionability,
- relevance, safety
- pass / revise

revise (max 1) with judge issues

validated JSON response -> UI

LLM: used by agents 1, 3, 5, 6, 7 when configured · Feedback: re-ranks agent 2 and orders actions in agent 6 · Monitoring: wraps every API call

③ CROSS-CUTTING SERVICES

LLM Provider (optional, pluggable)
- OpenAI gpt-4o-mini · Azure OpenAI
- Gemini 2.5 Flash · Claude Haiku 4.5
- no key -> deterministic offline mode

Feedback Loop (SQLite)
- ratings, helpful cases, effective actions
- -> case re-rank boost, action ordering
- -> category corrections = new labels

Monitoring & Observability
- structured logs + request IDs
- p50/p95 latency, errors, events
- alerts: latency, error rate, P1 burst

Evaluation
- DeepEval: relevancy, faithfulness, GEval
- retrieval: P@5, nDCG@10, MRR; models
- pytest: 60+ unit / API / agent tests